

تعالى نتكلم شوية عن Solid Principles

مجموعة من المبادئ التي تحسن جودة الـ **Software** عن طريق كتابة كود **clean** سهل الصيانة والتطوير

هم خمس مبادئ اتعملوا في البداية في الـ **OOP** ومع الوقت بقى بيتم تطبيقهم في السوفتوير كله.
غرضهم الأساسي انه يخلي السوفتوير اللي انت بتبنيه احسن وأقوى، بصيغة ثانية يكون **Solid** اكتر

=====

أول مبدأ هنتكلم عنه النهاردة في حرف الـ **S** هو الـ **Single Responsibility**

\$ SRP له اكتر من تعريف او شرح، لكن كلهم بينصبوا حول نفس الفكرة بالظبط، وهي إن الحاجة تعمل شيء واحد فقط وبالتالي السبب لتغييرها هو سبب واحد فقط.

*** SRP** معناها ان كل **Function** او **Class** او **Model** له مسئولية واحدة فقط يقوم بيها وله سبب واحد فقط في التغيير

اي هو الـ **Single Responsibility**؟؟

تعالى نعرف الاول هو اي وبعدين ناخذ امثلة

◆ المبدأ دا كل ال عايزه منك انك تخلي كل **class** مسئول عن وظيفة واحدة بس .. وبكدا يكون فيه سبب واحد ممكن تغير او تعدل ال **class** عشانه

--تعالى نوضح اكتر بمثال

← لو فيه **class** مسئول انه يضيف **Employee** ف ال **database** وبعدين بيعتله **email** انه تم اضافته

-ف هنا ال **class** المفروض يكون مسئول عن حاجة واحدة بس وهي انه يضيف ال **Employee**

✕ لكن انه بيعتله ايميل فدي كدا حاجة ثانية المفروض يكون فيه **class** ثاني وليكن **EmailService** مسئول عن الحوار دا

✓ وال Class المسئول عن الاضافة يقدر ينادي ع Method SendEmail من Class ال EmailService عادي بعد ما يضيف ال Employee عشان بيعتله الايميل

✓ كذا انت خلّيت كل Class مسئول عن وظيفة واحدة بس .. واحد مسئول انه يضيف والتاني مسئول انه بيعت الايميل

طب استفتت كذا اي لما طبقت المبدأ دا ؟؟

◆ كذا انت قسمت Proget لأجزاء صغيرة وبالتالي لو حصل اي مشكلة هتوصلها بسهولة عشان تحلها

◆ كمان كل Method لما تبقي مسئولة عن حاجة واحدة بس فانت تقدر تعيد استخدامها ف اكثر من مكان

◆ دا غير انه هيقفل اعتماد ال Classes ع بعض بالتالي لما اعدل ف مكان مش هلاقي مكان تاني وقع ويحصل اخطاء:

. الكلاس ده بيعمل حاجتين: يضيف موظف، وبيعته إيميل.

. لو حصل تعديل في طريقة الإيميل → هنعديل في كود الإضافة كمان. وده مش صح.

```
✖ : الكود الغلط

public class EmployeeService
{
    public void AddEmployee(string name, string email)
    {
        // إضافة الموظف
        // إرسال الإيميل
    }
}
```

✓ هنعمل برنامج صغير يضيف موظف (Employee) وبيعته له إيميل، بس هنقسم المسئوليات عشان نطبق SRP صح.

1. كلاس خاص بإضافة الموظف

```
public class EmployeeRepository
{
    public void Add(string name)
    {
        Console.WriteLine($"Added {name} to database");
    }
}
```

2. كلاس خاص بإرسال الإيميل

```
public class EmailService
{
    public void Send(string email)
    {
        Console.WriteLine($"Email sent to {email}");
    }
}
```

EmployeeService : مسؤول
ينادي الاثنين مع بعض، من غير
ما يتحمّل تفاصيل كل واحد..

```
public class EmployeeService
{
    private EmployeeRepository _repository = new EmployeeRepository();
    private EmailService _emailService = new EmailService();
    public void AddEmployee(string name, string email)
    {
        _repository.Add(name);
        _emailService.Send(email);
    }
}
```

الميزة هنا: 

- بننشئ objects مرة واحدة بس.
- ونستخدمهم في أي عدد من الدوال.
- الكود أوضح وأنصف وأسهل في التعديل.

```
private EmployeeRepository _repository = new EmployeeRepository();
```

```
private EmailService _emailService = new EmailService();
```

🎯 السؤال: ليه بنكتب السطرين دول أصلاً؟

الجواب ببساطة:

لما نكتب السطر ده، إحنا بننشئ (Object) من EmployeeRepository ونسخة من EmailService وبنخزن النسخة دي جوا متغير خاص (private variable) داخل الكلاس.

نقدر نستخدم الكائنات دي (EmployeeRepository) و (EmailService) في أي دالة جوه الكلاس.

بدل ما نكرر اكرت object كل مرة، بنكريته مرة واحدة ونخزنه في متغير خاص.

لو ما استخدمناش المتغيرات:

كل مرة تستدعي الدالة AddEmployee، هيتم:

- إنشاء EmployeeRepository جديد.
- وإنشاء EmailService جديد.

🔗 ده اسمه تكرار غير ضروري → ملوش فائدة، وبيصعب تعديل الكود بعدين.

✗ مثال من غير المتغيرات الخاصة

```
public void AddEmployee(string name, string email)
{
    new EmployeeRepository().Add(name);
    new EmailService().Send(email);
}
```

EmployeeRepository :: مسئول عن الإضافة بس

EmailService :: مسئول عن الإيميل بس

EmployeeService :: يستخدم الاثنين مع بعض بدون ما يحمل مسئوليات إضافية

```
class Program
{
    static void Main(string[] args)
    {
        EmployeeService service = new EmployeeService();
        service.AddEmployee("Ali", "ali@example.com");
    }
}
```

اي هو **Open-Closed Principle (OCP)** ؟ 😊 معناه ان بيسمح بالاضافه وغير مسموح بالتعديل

الكود بتاعك لازم يكون **Open** عشان تضيف فيه **Methods** جديدة

لكن **Closed** بحيث متغيرش في الجزء اللي شغال كويس

هنتكلم ف المقالة دي عن ثاني مبدأ من مبادئ ال **SOLID** وهو ال **open-closed principle**

--لو عندك **Class** اسمه **NotificationService** ودا من اسمه مسئول انه بيعت

Notification ف هيكون فيه **Attributes** زي ال **Message** ودي هتكون **String** وفيها الرسالة ال هتتبع وكمان ال **type** عشان نعرف نوع **Notification** ال هتتبع وفانكشن **Send** مسئولة انها تبعت ال **Notification** بس الاول بتشوف النوع لو هتتبع ع ال **Email** ولا مثلاً **Sms** او غيره ودا هيعمل ب **if .. else if** وكدا

✗ يعني كدا كل ما تحتاج تضيف نوع **Notification** جديد فانت هتضطر تروح ع ال **Class** وتعدل ف ال **function** بتاعته بأنك تزود **if else** جديدة وهكذا ودا مش صح

فيركز على جعل الكود مفتوحاً للتوسع ومغلقاً للتعديل **OCP**

مسموح بالتوسع والاضافه **Open > Extended**
غير مسموح بالتعديل علي الكود الحالي **Close > Modified**

❌ إيه المشكلة هنا؟

كل مرة تضيف نوع جديد من **Notifications**، لازم تدخل جوه الكود وتعديل.

أي تعديل ممكن بيؤوظ حاجة شغالة.

صعب تختبر.

الكود بيكبر بسرعة ويبقى معقد.

```
public class NotificationService
{
    public void Send(string type, string message)
    {
        if (type == "Email")
        {
            Console.WriteLine("Sending Email: " + message);
        }
        else if (type == "SMS")
        {
            Console.WriteLine("Sending SMS: " + message);
        }
        else if (type == "WhatsApp")
        {
            Console.WriteLine("Sending WhatsApp: " + message);
        }
        // ... وهانذا
    }
}
```

--طب هنحل المشكلة هنا ازاي؟؟

✅ المشكلة هنا هتتحل لو كنت مطبق ال **OCP** من الاول - 😊

💎 طب اي هو دا وازاي اطبقه ؟

المبدأ دا بيقولك ان اي **Class** او فانكشن عندك المفروض انها تكون مقفولة عن اي تعديلات .. ولكن بيبقى مفتوح عادي لأي اضافات جديدة

💎 والمبدأ دا بيطبق عن طريق ال **Abstract Class** او ال **Interface** تعالي نشوف ازاي بأننا نحل المشكلة ال قابلتنا فوق

✅ هنخلي ال **Notification** كلاس يكون .. **Abstract** بأن ال **Send** فانكشن مش هيبقي جواها اي كود وكم ان هنشيل ال **type attribute** لأننا مش هحتاجها ونعمل كلاس جديد وليكن

EmailNotifier ونخليه يورث من كلاس ال **Notification** وبكدا هو لازم يكتب الكود بتاعت ال **Send** فانكشن ع حسب بقي هو هيبعت ازااي وكلاس كمان **SMSNotifier** وبرضو هيبورث من كلاس ال **Notification** ويكتب الكود بتاع ال **Send** فانكشن زي ما هو عايز

✓ يبقى كدا كل ما احتاج اضيف نوع جديد انا مش هعدل ع اي كود كان موجود قبل كدا لا هعمل بس **Class** جديد للنوع دا واخليه يورث من **Class** ال -- **Notification** طب المبدأ دا كدا فادني ب ايه ؟؟ ✨ المبدأ دا من اهم المبادئ ال هتخليك تضيف **Features** جديدة من غير ما تعدل ع الكود القديم لأنك لو عدلت ممكن تحصل مشاكل ف اماكن تانية دا غير انه هو كمان هيدعم المبدأ ال **Single Responsibility** فاهو ال

أ. نعمل كلاس (Abstract) أو: **Interface**

```
public abstract class Notification
{
    public abstract void Send(string message);
}
```

ب. نعمل كلاس لكل نوع **Notification**

خلي بالك جوه ال **method** اللى هي **send**

جوها **Atrbute** اللى هي **string message** علشان نسيبتها

```
public class EmailNotification : Notification
{
    public override void Send()
    {
        Console.WriteLine("Email sent: " + Message);
    }
}

public class SMSNotification : Notification
{
    public override void Send()
    {
        Console.WriteLine("SMS sent: " + Message);
    }
}
```

ج. نستخدم كلاس بيرسل الإشعار:

```
public class NotificationService
{
    public void SendNotification(Notification notification)
    {
        notification.Send();
    }
}
```

Email sent: Welcome Email!

SMS sent: Welcome SMS!

```
class Program
{
    static void Main()
    {
        NotificationService service = new NotificationService();

        Notification email = new EmailNotification { Message = "Welcome Email!" };
        Notification sms = new SMSNotification { Message = "Welcome SMS!" };

        service.SendNotification(email);
        service.SendNotification(sms);
    }
}
```

Liskov Substitution Principle (3)

مبدأ **Liskov Substitution** يقول: يجب ان تكون **Childs Classes** قادرة على ع ان
تحل محل **Pernt Classes** دون ان تؤدي الى كسر او تغيير في سلوك ال **Code**

يعني لو عندك **Perent Class** وورثت منه **Child class** يبقى لازم تقدر تستخدم **Child Class** مكان **Perent** من غير ما يحصل مشاكل او تغيير ف سلوك البرنامج

يعني ال **Child** لازم يشتغل زي من **Perent** من غير مفاجات

طب مثال ثاني: تخيل عندك عربية كبيرة وعربية صغيرة. الاثنين دول عربيات، يعني ليهم نفس الـ **Properties** (عجل، موتور، فرامل). لو مكان ما بيقبل بس العربيات الكبيرة، تقدر تدخل العربية الصغيرة بدالها عادي، لأنها برضه عربية

مثال في الـ **Code**:

* الغلط هنا استخدمت **Soap** مكان **juice** لانهم نفس العيلة لكن سلوك الـ **Soap** مش زي الـ **Perent** بيكسر البرنامج ودا كذا كسر مبدأ **Liskov Substitution**

```

    مثال فيه مشكله بيكسر مبدأ LSP
public class juice
{
    public virtual void drink()
    {
        Console.WriteLine("you drink juice");
    }
}

public class orangejuice : juice
{
    public override void drink()
    {
        Console.WriteLine("you drink juice");
    }
}

class soap:juice    // المشكله هنا
{
    public override void drink() // لان ده صابون ي مجنون مش بيتشرب
    {
        throw new Exception("ostrich cant fly");
    }
}

private static void Main(string[] args)
{
    juice b = new soap(); // استبدلنا الاب بالابن
    b.drink(); // البرنامج ضرب لان الصابون مش بيتشرب
}
```

{ الحل الصحيح }



```
1: interface IDrinkable { void Drink(); }

2: class juice : IDrinkable
{
    public void Drink()
    {
        Console.WriteLine("you drink juice");
    }
}

3: class orangejuice : juice
{
    public void Drink()
    {
        Console.WriteLine("you drink juice");
    }
}

4: class soap
{
    public void wash() // لان ده صابون ي مجنون مش بيتشرب
    {
        throw new Exception("you wash your hands withe soap ");
    }
}

private static void Main(string[] args)
{
    IDrinkable i = new juice();
    i.Drink();
    IDrinkable ii=new orangejuice();
    ii.Drink();

    IDrinkable soap = new soap(); // الصابون مش بيتشرب فمش اقدر اكتبه كذا
}
```

اللى بيتشرب طبق مبدء IDrinkable واللى مشييتشرب زى Soap ملهوش علاقة

Interface Segregation (4)

"بينص على الكلاسات ماينفعش تُجبر على تنفيذ (methods) مش بتحتاجها".

بمعنى، بدل ما أعمل Interface كبير فيه methods كتير غير مرتبطة ببعض، بقسمه لـ Interfaces صغيرة ومتخصصة، وكل Class تطبق بس اللي هي محتاجاه.

ده بيخلي الكود أبسط، أسهل في الفهم، وبيققل التغيير غير الضروري لما يحصل تعديل أو إضافة.

(مثال كذا صغير)

تصور نفسك بتعمل أكل، عندك طباخ بيعمل الرز، وطباخ تاني بيعمل اللحم، وطباخ ثالث بيعمل السلطة. كل واحد متخصص في حاجة معينة. لو كل طباخ لازم

يعمل كل الأكل، هيبقى الأكل مش مضبوط وهياخد وقت كثير

بساطة، مبدأ **Interface Segregation** بيقول:

ما تخلّش العميل يتعامل مع حاجات مش محتاجها. خلي كل **Interface** مسئولة عن جزء محدد من الشغل. ليه ده مهم؟

بيققل التعقيد: كل **Interface** بتكون مسئولة عن جزء بسيط، فبيسهل فهم الكود وتطويره.

• بيزيد المرونة: لو عايز تضيف **method** جديدة، هتضيفها في **Interface**

المناسبة من غير ما تغير في باقي النظام.

بيحسن الصيانة: لو فيه مشكلة في جزء معين، هتقدر تصلحها بسرعة من غير ما تؤثر ع باقي الاجزاء

مثال ::

✗ المشكلة **Class User**: مضطر

يطبق دوال مش محتاجها

(**sendEmail**, **log**) وده ضد مبدأ

Interface Segregation.

```
interface IUser {
    void login();
    void register();
    void sendEmail();
    void log();
}

class User implements IUser {
    public void login() {
        // login logic
    }

    public void register() {
        // register logic
    }

    public void sendEmail() {
        // User محتاج الدالة دي
    }

    public void log() {
        // User محتاج الدالة دي
    }
}
```

✓ الكود بعد إعادة التنظيم والتقسيم الصح:

الخطوة 1: تقسيم الـ Interfaces ❄️

```
// Interface عن إدارة المستخدم
interface IUser {
    void login();
    void register();
}

// Interface لإرسال الإيميلات
interface IEmail {
    void sendEmail();
}

// Interface لتسجيل الأحداث أو الأخطاء
interface ILogger {
    void log();
}
```

الخطوة 2: تطبيق كل Class للـ Interface التي يحتاجها فقط ❄️

```
// Class عن المستخدم فقط
class User : IUser {
    public void login() {
        // login logic
    }

    public void register() {
        // register logic
    }
}
```

```
// Class مسؤول عن إرسال الإيميلات فقط
class EmailService : IEmail {
    public void sendEmail() {
        // send email logic
    }
}
```

```
// Class المسؤول عن اللوج فقط
class LoggerService : ILogger {
    public void log() {
        // log logic
    }
}
```

ملاحظات: 

- كل كلاس يطبق بس الـ **Interface** اللي بيحتاجها.
- لو حصل تغيير في **IEmail** مش هياثر على **User** لأن مفيش بينهم علاقة مباشرة.
- ده بيخلي الكود مرن، نظيف، قابل للتوسعة والصيانة بسهولة.

Dependency inversion (5)

المبدأ ده بيقول ببساطة:

✓ " ما تخليش كلاس يعتمد على كلاس تاني بشكل مباشر، خليه يعتمد على واجهة (interface) بدلاً من كده".

👁️ له؟ علشان لو الكلاس اللي تحت اتغير، الكلاس اللي فوق مايتأثرش!

📦 تخيل معايا المثال ده:

● كود غلط مافيهوش: (Dependency Inversion)

```

public class ClassA
{
    public void MethodA()
    {
        Console.WriteLine("أنا Class A");
    }
}

public class ClassB
{
    ClassA a;

    public void MethodB()
    {
        a = new ClassA(); // ← ClassB يخلق ClassA بنفسه!
        a.MethodA();      // ← يعتمد عليه تمامًا!
    }
}

```

في هذا الكود، **ClassB** يقوم بإنشاء **object** من **ClassA** داخل **MethodB**، مما يعني أن **ClassB** يعتمد بشكل مباشر على **ClassA**.

& المشكلة:

ClassB يعتمد على **ClassA** بشكل مباشر، وهذا يُعد انتهاكًا لمبدأ عكس **(Dependency Inversion)** (يعني "مرتبط بيه ارتباط مباشر"، ومينفعش **ClassB** يشتغل من غيره. بل يجب أن يعتمد على **(interface أو abstract)**)

🎯 طيب إيه المشكلة؟ ليه ده غلط؟

المشكلة إنك كده قيدت **ClassB**، وبقيت تقول:

"أنا مش هعرف أشتغل إلا بـ **ClassA**، ومينفعش أشتغل بحد غيره".

فالو:

- **ClassA** اتغير أو اتحذف أو اتبدل بكلاس تاني

✗ لازم تغير **ClassB** كمان

وده بيخلي الكود مربوط ببعضه جدًا، وده اسمه **Tightly Coupled**

طيب نعمل إيه؟

نستخدم Dependency Inversion Principle

يعني نخلي ClassB يقول:

"أنا مش هتعامل مع ClassA نفسه، أنا هتعامل مع حاجة " (interface) "اسمها...IClassA ومش فارق مين اللي يحققها ClassA أو غيره".

كود صح فيه: (Dependency Inversion)

1. نعمل Interface

```
Interface نعمل
public interface IClassA
{
    void MethodA();
}

نخلي ClassA يطبق Interface
public class ClassA : IClassA
{
    public void MethodA()
    {
        Console.WriteLine("لنا Class A");
    }
}
```

3. نخلي ClassB يعتمد على (interface) بدل من الكلاس:

```
public class ClassB
{
    IClassA a;

    // Constructor Injection هستخدم
    public ClassB(IClassA a)
    {
        this.a = a;
    }

    public void MethodB()
    {
        a.MethodA(); // بيشغلها من غير ما يعرف هي أنهي كلاس بالضبط
    }
}
```

ونشغل الكود كده:

```
class Program
{
    static void Main(string[] args)
    {
        IClassA a = new ClassA(); // أو أي حاجة ثانية ClassA2 ممكن في المستقبل نستخدم
        ClassB b = new ClassB(a); // بنعته الكلاس من برة
        b.MethodB(); // MethodA هينفذ اللي جوه
    }
}
```

ClassB مش بيعرف حاجة عن **ClassA**

🚧 الآن **ClassB** يعتمد على التجريد (**IClassA**) وليس على **ClassA** بشكل مباشر.

يعني بيقول:

أنا مش فارق معايا مين اللي ينفذ، المهم يكون فيه **Method** اسمها " (**MethodA**)

💡 جملة تحفظها للإنترفيو:

classB "بيعتمد على **abstraction** مش **implementation**، وده بيسهل التعديل، والاستبدال، واختبار الكود، وبيخلي النظام **loosely coupled**."

خلونا النهارده نتكلم عن الـ **SOLID PRINCIPLES**

ودي عبارة عن مبادئ معينة بنمشي عليها واحنا بنكتب الكود بتاعنا عشان يبقى الكود قابل للتوسع ومنسق وسهل التعامل معاه واي مطور يقدر يشارك في التطوير بدون مشاكل علي عكس الـ **Patterns Design** اللي هي حلول لبعض المشاكل مش مبادئ معينة لكتابة الكود بتاعنا

خامس وآخر مبدأ في ال SOLID PRINCIPLES حرف ال D وهو ال Dependency inversion.

- مبدأ ال Dependency inversion ده زي ما يكون عندك محل عربيات. بدل ما العامل اللي بيركب الموتور يكون عارف كل حاجة عن كل قطع العربية، احنا بنقوله "انت بس ركب الموتور، متقلقش من باقي الحاجات". يعني كل جزء في العربية بيعمل شغله من غير ما يتدخل في شغل الثاني. طيب مثال ثاني:

- تخيل نفسك عايز تعمل كوباية قهوة. أنت مش هتروح المزرعة تجيب بنفسك حبوب البن وتطحنها وتشويها وتسلقها. لا، أنت هتروح الكافيه وتطلب كوباية قهوة جاهزة عادي جدا. الكافيه هو اللي بيعمل كل الشغل التفصيلي ده، وأنت بس بتطلب المنتج النهائي.

في المثال اللي موجود في الصورة

لما نقسم الكود كده، هنلاقي عدة مميزات:

سهولة التغيير: لو عايز تغير نوع القهوة، هتغير بس في Class اللي بيشتغل على القهوة، من غير ما تغير في أي حاجة ثانية.

إعادة الاستخدام: ممكن تستخدم نفس ال Interface (الواجهة) ICoffeeMaker في أماكن ثانية في البرنامج.

اختبار أسهل: كل جزء من الكود بيبقى مستقل عن الثاني، فسهل تختبره لوحده.

ببساطة، مبدأ عكس التبعية بيقول: الكود الكبير والمهم (زي Customer) مش المفروض يعرف التفاصيل الصغيرة (زي طريقة عمل القهوة). الكود الكبير بيعرف بس الواجهة اللي بيطلب منها الخدمة.

بإختصار المبدأ ده بيتلخص في الجملة ديه:

**High-level modules should not depend on low-level modules
Both should depend on abstractions**

// مثال سي: الكود الكبير يعتمد على الكود الصغير مباشرة

```
class CoffeeMaker {  
    public void MakeCoffee() {  
        // هنا يكتب كل خطوات عمل القهوة من البداية للنهاية  
        // ...  
    }  
}
```

// مثال جيد: الكود الكبير يعتمد على واجهة

```
interface ICoffeeMaker {  
    void MakeCoffee();  
}
```

```
class CoffeeMachine : ICoffeeMaker {  
    public void MakeCoffee() {  
        // هنا يكتب كل خطوات عمل القهوة من البداية للنهاية  
        // ...  
    }  
}
```

```
class Customer {  
    private ICoffeeMaker _coffeeMaker;  
  
    public Customer(ICoffeeMaker coffeeMaker) {  
        _coffeeMaker = coffeeMaker;  
    }  
  
    public void OrderCoffee() {  
        _coffeeMaker.MakeCoffee();  
    }  
}
```